

Progetto “Rilievi e Perizie”

Una certa azienda di assicurazioni ha necessità di archiviare in tempo reale su un server le fotografie di rilievi / perizie eseguite dai propri dipendenti. Il DB, sarà costituito da 2 collezioni: utenti e perizie.

La collezione **utenti**, anche in presenza degli _id creati da mongo, utilizza come chiave di riferimento lo username rappresentato dall'indirizzo mail dell'utente. Questa collezione (oltre all'_id) contiene sostanzialmente solo tre campi:

- username
- password cifrata in bcrypt
- info aggiuntive sull'utente (che può anche essere vuoto)

La collezione **perizie** deve contenere :

- **Codice identificativo della perizia**
- **Un campo di descrizione generale dell'intera perizia**
- **Un vettore enumerativo di fotografie, ciascuna con un proprio commento aggiuntivo**
- **Username dell'operatore che ha eseguito la perizia**
- **Data e ora della perizia**
- **Coordinate geografiche del luogo relativo alla perizia**

L'applicazione sarà costituita da **tre parti**:

1) Applicazione Server realizzata con Express

L'applicazione non fornisce pagine ma fornisce **solo servizi** per entrambe le applicazioni successive.
Implementa il protocollo JWT per la validazione degli utenti.
Gestisce l'interfaccia con Cloudinary per l'upload delle fotografie

2) Applicazione ADMIN realizzata con Angular utilizzabile dall'ufficio

Funge da supervisione per l'ufficio centrale e consente al responsabile dell'ufficio di visualizzare tutte le varie perizie uploadate dagli operatori.

L'utente **ADMIN** è l'unico autorizzato ad accedere ed utilizzare questa applicazione

Solo ADMIN può creare nuovi utenti abilitati all'esecuzione delle perizie. La creazione di nuovi utenti può essere gestita in due modalità diverse.

- Si può demandare il problema al login with google. Il server gestisce all'interno di un file testuale esterno un elenco di username abilitati. L'amministratore, in qualunque momento, potrà modificare questo elenco aggiungendo nuovi utenti. Nel momento in cui l'utente eseguirà il login with google, il server controlla la presenza dello username all'interno della lista. Se lo username è valido, allora crea un nuovo token jwt con durata anche lunga per NON dover rieseguire ogni volta il login. L'utente si autenterà sempre solo tramite Login With Google. **Occorre verificare come gestire il Login With Google da una applicazione Andorid**
- L'amministratore registra manualmente il nuovo utente tramite apposita pagina di registrazione in cui inserisce username ed eventuali informazioni aggiuntive. La password viene generata casualmente dal server (con lunghezza 10 caratteri) e viene inviata tramite mail al nuovo utente.

E' possibile anche prevedere una modalità mista: alcuni utenti si autenteranno tramite Login With Google, altri (inseriti dall'amministratore) si autenteranno mediante username e password.

L'utente ADMIN può visualizzare su una unica **mappa** (aperta con centro nella sede dell'ufficio) tutte le perizie eseguite dagli operatori. Ogni perizia sarà caratterizzata da un apposito segnaposto posizionato sul luogo della perizia. In corrispondenza del click sul segnaposto verranno visualizzati tutti i dettagli della perizia (descrizione, immagini, etc)

- Un apposito filtro consente di scegliere un utente e visualizzare soltanto le perizie eseguite da quell'utente
- L'utente ADMIN può modificare sia la descrizione generale della perizia sia gli eventuali commenti alle singole immagini
- Per ogni perizia l'utente ADMIN può visualizzare percorso e tempo di percorrenza per il raggiungimento del luogo in cui è stata svolta la perizia, a partire dalla sede dell'ufficio (che può essere ad esempio il vallauri o altro luogo)

3) APP android

Realizzata tramite angular / ionic viene buildata tramite capacitor ed installata sul telefono degli operatori. La app utilizzerà anche lei gli appositi servizi creati sul server express.

L'operatore, al termine dello svolgimento della perizia, dovrà uploadare sul server remoto tutte le informazioni relative alla perizia stessa, cioè in pratica la **descrizione generale** e le varie **fotografie** con i relativi commenti.

Coordinate GPS, Data e Ora e Codice Operatore verranno aggiunte in automatico dal sistema.

Le immagini possono essere salvate all'interno del DB in formato **base64** (con impostazione di una dimensione massima molto ridotta) oppure, **molto meglio**, su un file server esterno (**cloudinary**)

La app può eseguire un unico upload oppure uploadare prima il record principale e dopo via via le varie fotografie con i relativi commenti

L'operatore, al primo utilizzo, deve eseguire il login secondo una delle modalità indicate in precedenza.

Occorre verificare il funzionamento dei cookies all'interno di una app android

3) APP IOS

Chi non ha un telefono android può eseguire un build per IOS. A tale scopo è però necessario l'utilizzo di un pc MAC

Piano di Lavoro

- 1) Creare su Compass il database costituito dalle 2 tabelle UTENTI e PERIZIE con due record ciascuna
- 2) Realizzare il server express e l'applicazione admin per visualizzare le 2 perizie create staticamente nel DB. Pubblicarla su render
- 3) Creare la app ionic da utilizzare sugli smartphone, con i relativi servizi su express
- 4) La pubblicazione del server su **render** è facoltativa

Note

- Utilizzando i servizi di **Login with Google** e/o **Mail with OAuth**, l'applicativo finale **sfora i 512 MB** di **render** che segnala un errore di sfioramento della heap. Senza questi servizi la pubblicazione è ok. Eventualmente utilizzare BUN che è una versione 'leggera' di nodejs. Utilizza le STESSA identiche librerie che possono essere installate tramite BUN INSTALL. Per lanciare una applicazione typescript digitare BUN server.ts. Anche su render il comando di lancio sarà BUN server.ts
- Con render / cloudinary l'upload delle foto è decisamente lento. Impostare un timeout di **30 sec.**
- Per la presentazione utilizzare un **cavo di rete** con adattatore e non il WiFi Vallauri che è lentissimo. E' sufficiente che PC e smartphone siano collegati alla stessa rete locale.